IMD0904 –Plataforma de Hardware para Internet das Coisas

Prof. Daniel Lopes Martins

UFRN | metrópole DIGITAL

AULA 13

Comunicação entre dispositivos

Por que dispositivos precisam se comunicar?



[1]

- Quase tudo no mundo de hoje está intrinsecamente interligado transmitindo informações.

- Mais que conectar computadores ou celulares entre si, essa área permite conectar qualquer equipamento, desde um minúsculo sensor até um gigantesco servidor.

- Mas como essa troca de dados é realizada? O que está envolvido nisso?

Conceitos básicos

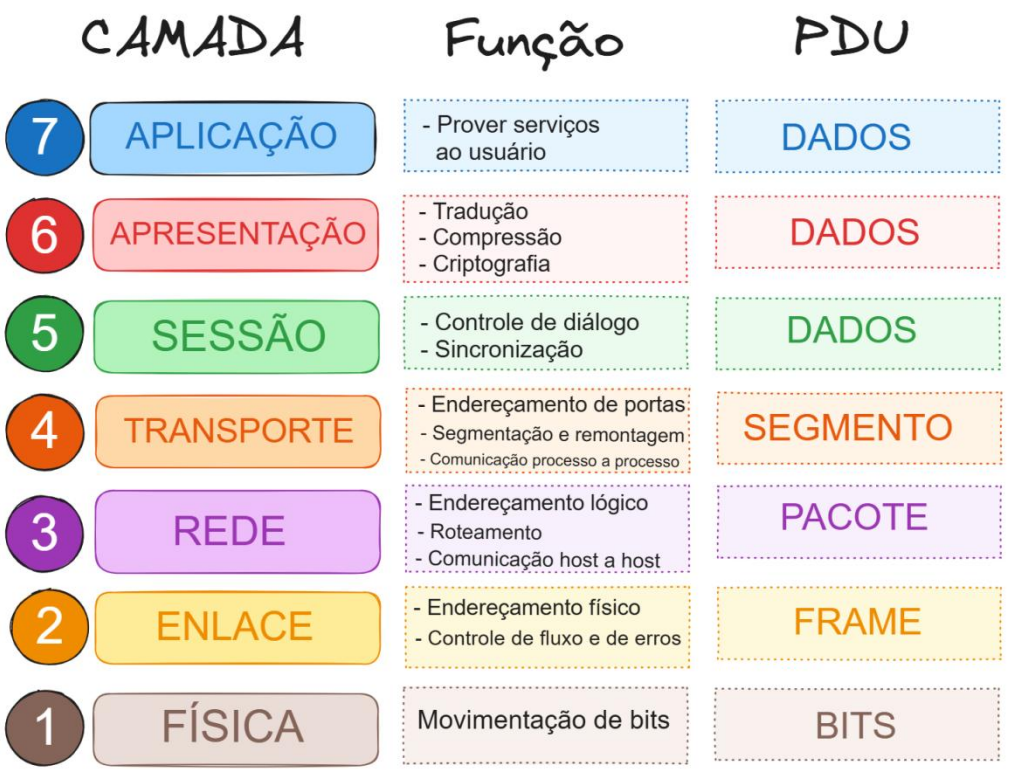
- O processo de comunicação envolve ao alguns elementos importantes:
  - A mensagem – A informação a ser compartilhada.
  - O transmissor e o receptor – Os dispositivos que vão interagir entre si.
  - O meio de transmissão – O caminho físico por onde a mensagem viaja.
  - O protocolo – É o conjunto de regras e padrões que garante “uma linguagem comum”, permitindo o entendimento entre os envolvidos.



[2]

Modelo OSI

- O Modelo OSI foi proposto pela ISSO como uma tentativa de padronizar os protocolos usados nas diversas camadas de comunicação.
- Dependendo do sistema o modelo pode ser simplificado.

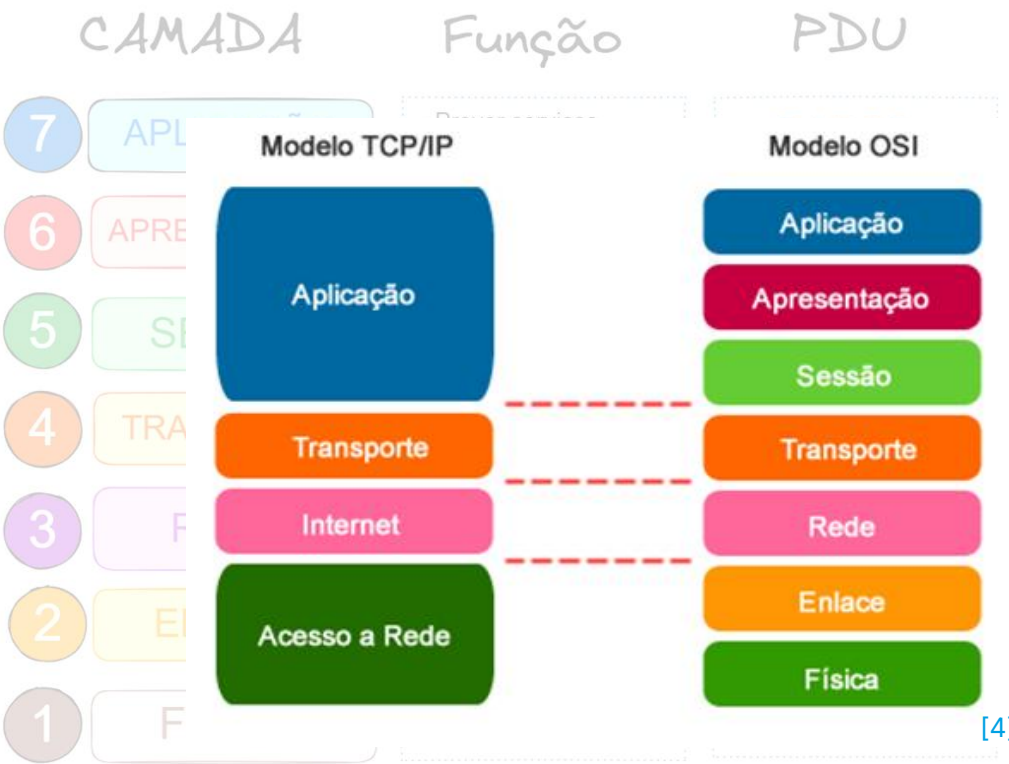


[3]

Modelo OSI

- O Modelo OSI foi proposto pela ISSO como uma tentativa de padronizar os protocolos usados nas diversas camadas de comunicação.

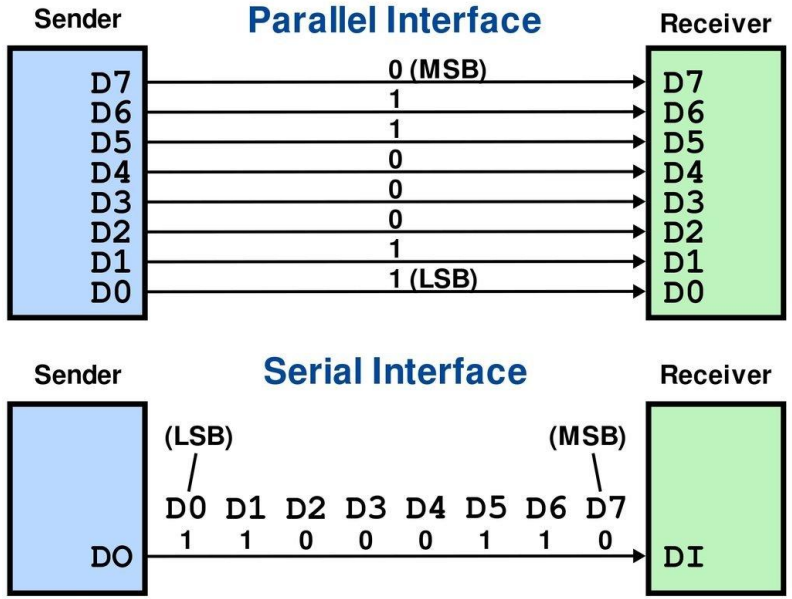
- Dependendo do sistema o modelo pode ser simplificado.



[4]

Formas de comunicação

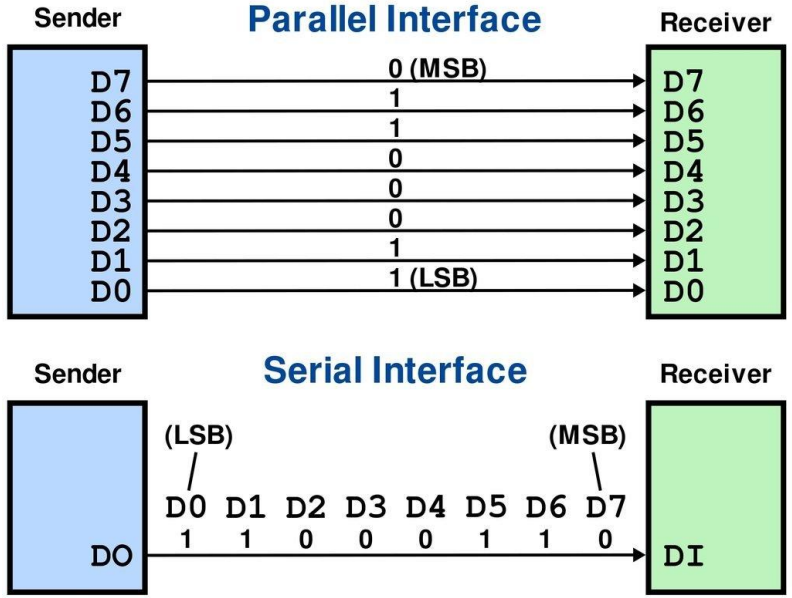
- Tipo de comunicação:
  - Paralela
    - Os bits transmitidos ao mesmo tempo (vários canais/fios).
    - Possui uma velocidade maior (em teoria).
    - Possui um custo mais elevado (espaço, material).
    - Limitada em distância (problemas de sincronismo).



[5]

Formas de comunicação

- Tipo de comunicação:
  - Serial
    - Os bits transmitidos por um único canal, de forma sequencial.
    - Mais lento (por natureza), mas pode ser mais rápida em distância.
    - Custo menor (um canal de transmissão).
    - Ideal para longas distâncias (menos suscetível a crosstalk, por exemplo).



[5]

Métodos de comunicação

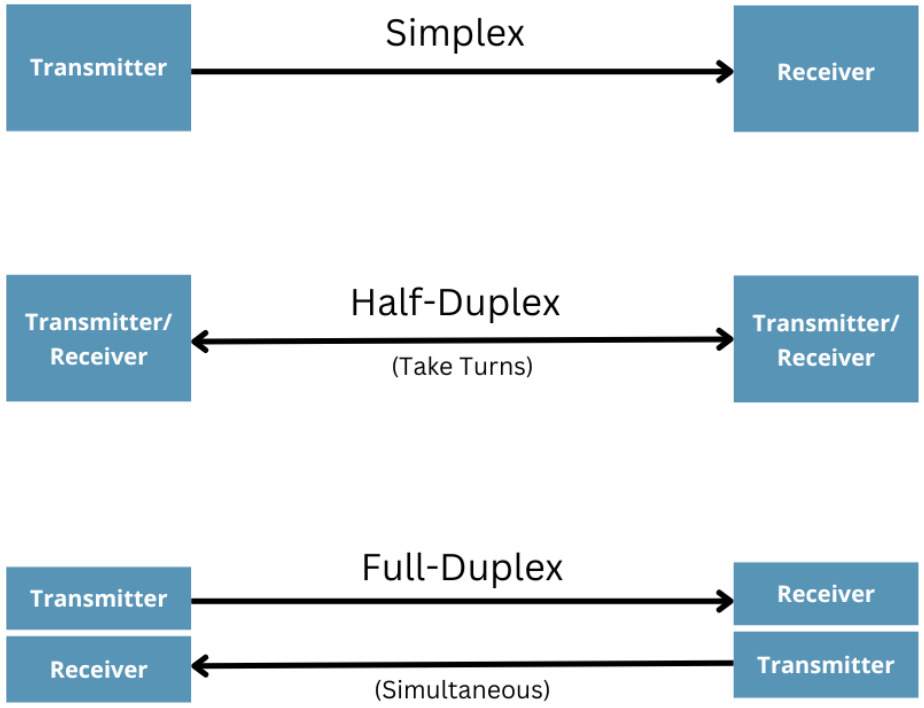
- Comunicação síncrona
  - Utiliza-se um sinal de clock ou outro método de temporização para sincronização entre dispositivos (normalmente usa uma linha dedicada a isso).
  - Dados enviados em blocos contínuos sem necessidade de interrupções.
  - Geralmente mais rápida, pois não tem overhead de bits para indicar início e fim de um pacote.
  - Mais complexo, pois requer lógica de clock e mecanismos de recuperação de sincronia (além de utilizar mais fios).

Métodos de comunicação

- Comunicação assíncrona
  - Transmissor e receptor não compartilham um sinal de clock comum, sendo a sincronização realizada com base no próprio fluxo de dados.
  - Dados enviados byte a byte ou caractere a caractere com possíveis lacunas de tempo entre cada envio.
  - Geralmente mais lenta, pois há overhead (bits adicionais de start e stop) para indicar início e fim de um pacote.
  - Mais simples de implementar e mais flexível na questão do tempo (mais tolerante a pequenas variações de tempo), além de mais robusto a longas distâncias.

Sentido de transmissão

- Modos
  - Simplex
    - Unidirecional, comunicação ocorre apenas em uma direção (transmissor ao Receptor).
  - Half-Duplex
    - Bidirecional, mas os dispositivo necessitam se revezar.
  - Full-Duplex
    - Bidirecional e os dispositivos podem transmitir e receber ao mesmo tempo.



[6]

Protocolos seriais mais comuns

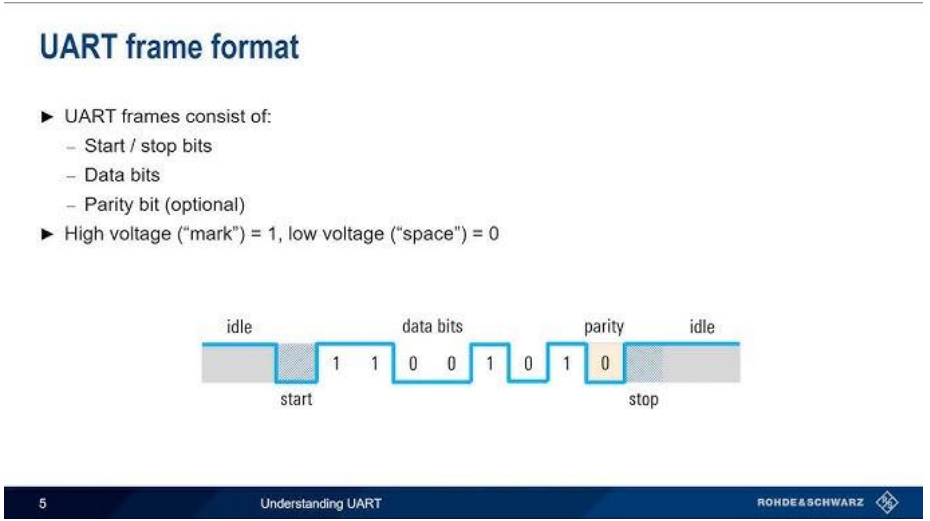
- Vamos abordar alguns protocolos seriais mais usados com MCUs.

- Alguns desses protocolos são nativos de alguns microcontroladores, outros são necessários hardwares externos.

- Uma observação importante, quando se fala de pinos de comunicação, isso se refere as vias de comunicação. Todos necessitam do pino de terra e alguns também do VCC.

Protocolos seriais mais comuns

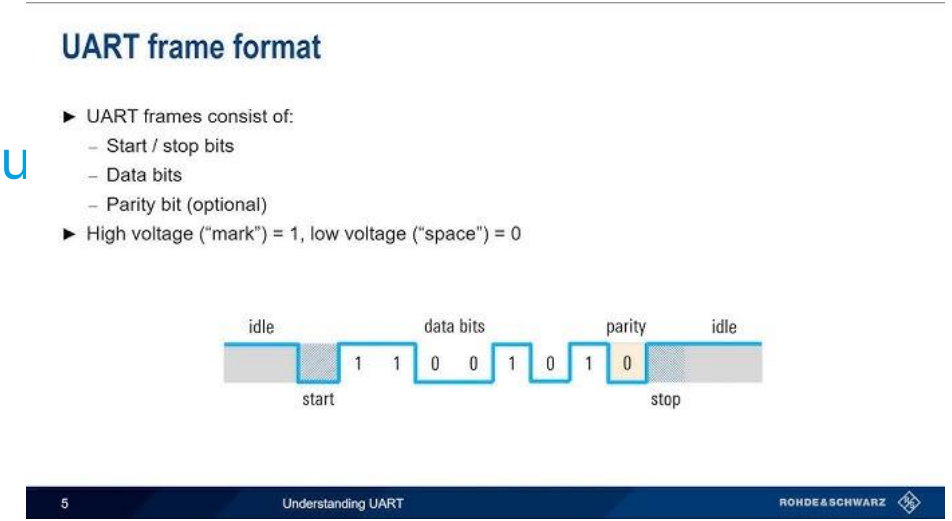
- UART – Sigla para Universal Asynchronouns Receive/Trasmitter. É um protocolo extremamente simples, full-duplex. Usa bits de start e stop, não requer sinal de clock e trasmite 8 bits por vez.
- Seu quadro é formado:
  - Start bit.
  - Bits de dados.
  - Bit de paridade
  - Stop bit.
- Alguns parâmetros devem ser configurados para uma comunicação correta.



[7]

Protocolos seriais mais comuns

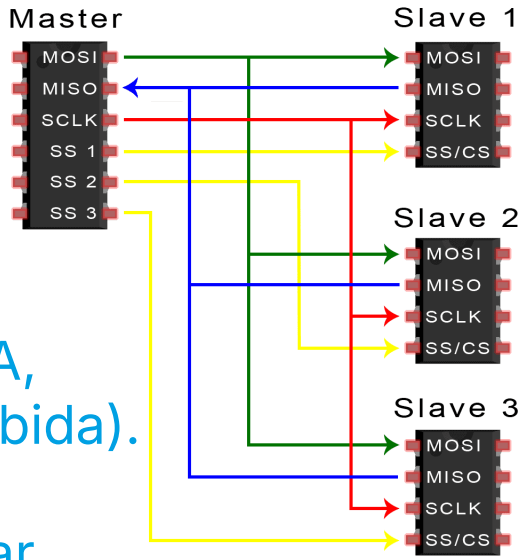
- Parametros do UART:
  - Baud Rate: taxa de transmissão, que deve ser idêntica tanto no transmissor quanto no receptor.
  - Bits de dados: pode ser configurado para trabalhar de 5 a 9 bits.
  - Bit de paridade: bit usado para a verificação de erros de transmissão. Pode ser par ou impar.
  - Stop bit: que pode ser 1, 1,5 ou 2 bits.



[7]

Protocolos seriais mais comuns

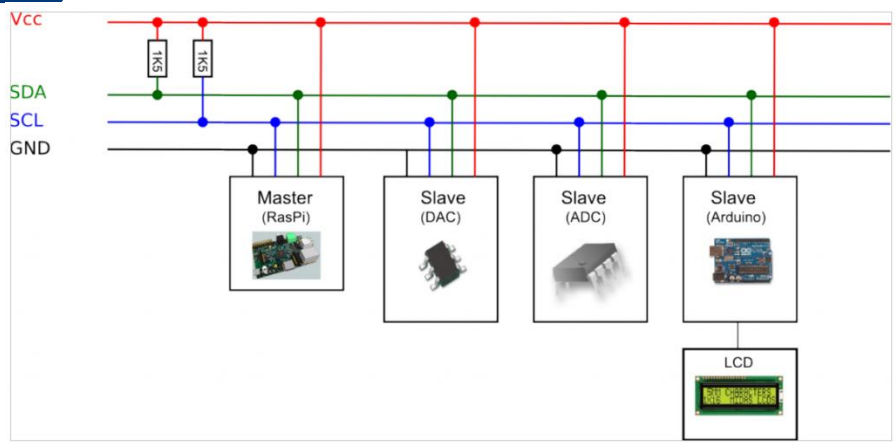
- SPI – Serial Peripherical Interface. É um protocolo de comunica síncrono baseado nos papéis de mestre/escravo. É full-duplex, mais rápido que o I^2C e UART.
- Para cada escravo conectado, deve existir uma linha SS.
- Parâmetros:
  - Modo SPI: 0, 1, 2, 3, relativo a repouso do clock e qual a borda do clock (CPOL e CPHA, respectivamente, exemplo: 0 – baixo, 0 – subida).
  - Configuração do prescaler para definir a frequência do mestre (escravo deve suportar.



[8]

Protocolos seriais mais comuns

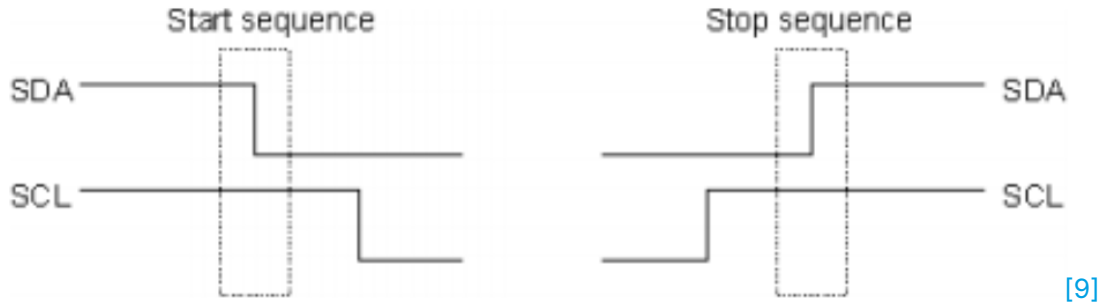
- I²C – Inter-Integrated Circuit. Prioriza simplicidade em quantidade de fios e capacidade de endereçamento.
- Utiliza apenas duas linhas de comunicação (SDA e SCL).
- Mestre-escravo, sendo o mestre a gerar o sinal de clock.
- Bidirecional hal-duplex.



[9]

Protocolos seriais mais comuns

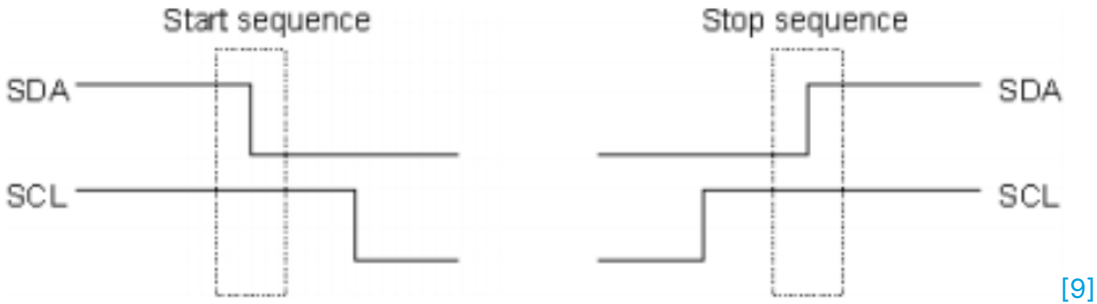
- Pode ter mais de um mestre por linhas e vários escravos.
- Usa endereçamento único para cada escravo de 7 a 10 bits.
- Linha com resistor de pull-up e pinos em dreno aberto.
- Usa uma sequência específica para iniciar e terminar uma transmissão.



[9]

Protocolos seriais mais comuns

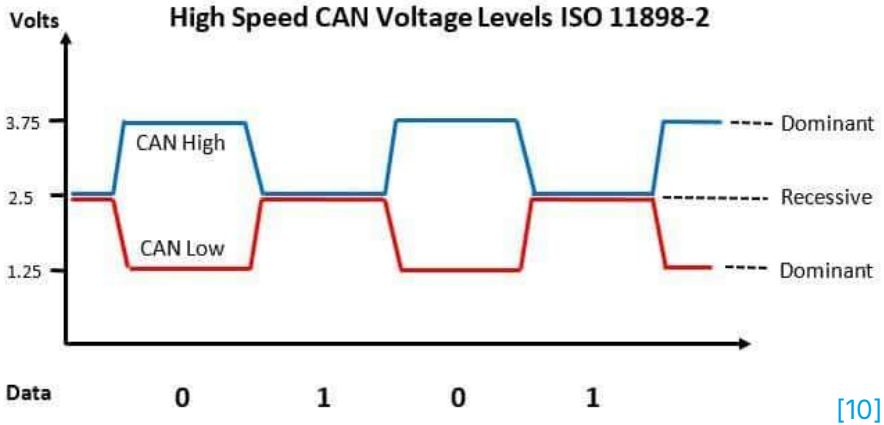
- Depois de avisar aos escravos que haverá transmissão, coloca endereço na linha.
- Escravo correspondente responde com um ack.
- Transfere os dados byte a byte, com acks no meio.
- Ao final, mestre indica o fim da transmissão.



[9]

Protocolos seriais mais comuns

- CAN – Controller Area Network. Protocolo de barramento serial síncrono ou assíncrono.
- Permite que dispositivos comuniquem-se entre si em aplicações sem a necessidade de um computador host.
- Usa CSMA/CD para acesso ao barramento.
- Half-Duplex sobre par trançado (CAN-H e CAN-L em sinal diferencial).
- Comunicação orientada a mensagens (com ID).



[10]

Protocolos seriais mais comuns

- O CAN possui um alto overhead (vários bits de controle/erros).
- Possui um hardware mais complexo por causa da arbitragem e detecção de erros.
- Possui uma velocidade limitada (mais lenta) se comparado ao SPI.
- Mas é um protocolo altamente imune a ruído e interferências (tensão diferencial).

Protocolos seriais mais comuns

- Alguns microcontroladores atualmente possuem outros tipos de hardware para realizar as comunicações:
  - Wi-Fi – Protoco sem fio que trabalha nas bandas de 2.4GHz e 5 GHz, com uso de CSMA/CA, e alta taxa de transmissão, permitidno conexão com a internet.
  - Bluetooth/BLE – Protocolo sem fio de curto alcance, de baixo consumo, projetado para conexão de dispositivos pessoais. Opera na faixa de 2.4GHz, possuindo 2 categorias, o classico e o BLE.
  - Zigbee – Protocolo de comunicação sem fio baseado na especificação 802.15.4, usado em ambientes de IoT e automação residencial (e as vezes industrial). Trabalha com baixas taxas de transmissão (250 Kbps).

Protocolos seriais mais comuns

- Alguns microcontroladores atualmente possuem outros tipos de hardware para realizar as comunicações:
  - Zigbee – Topologia em Mesh e por isso alta escalabilidade (até 65000 nós) e alta segurança com AES 128 bits.
  - Lora – Abreviação de Long Range. É uma tecnologia de comunicação sem fio de longo alcance e baixíssimo consumo de energia, ideal para IoT. A camada física do protocolo define a modulação e frequência de trabalho, enquanto a camada de rede define o protocolo de comunicação e arquitetura de rede (implementa a camada de rede, segurança e gerenciamento de dispositivos, em topologia estrela e com uso de gateway).

Referências complementares

- https://www.robocore.net/tutoriais/comparacao-entre-protocolos-de-comunicacao-serial.html?srsltid=AfmBOoqOCDIBfCJODbaIpInvj8Nxw8zhhr93yxFfrMF9hNWaGeRekKA6
- https://www.estrategiaconcursos.com.br/blog/modelo-osi-tse-ti/#
- https://blog.arduinoomega.com/entendo-o-protocolo-i2c/
- https://pt.pcm-cable.com/info/what-is-synchronous-and-asynchronous-in-can-88517716.html

Referências complementares

- https://embarcados.com.br/barramento-can-entre-arduinos-uno/

Fontes

[1] – https://blog.gokursos.com/carreiras/tecnologia/internet-das-coisas-no-futuro/
[2] – https://br.freepik.com/fotos-gratis/estudantes-inter-raciais-se-divertem-e-riem-alegremente-no-parque_9659401.htm
[3] – https://www.estrategiaconcursos.com.br/blog/modelo-osi-tse-ti/#
[4] – https://www.dltec.com.br/blog/redes/4-motivos-para-aprender-modelo-osi/
[5] – https://www.electricalelibrary.com/2023/09/02/protocolo-de-comunicacao-spi/
[6] – https://www.everythingrf.com/community/what-is-full-duplex
[7] – https://www.youtube.com/watch?v=sTHckUyxwp8
[8] – https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/
[9] – https://blog.arduinoomega.com/entendo-o-protocolo-i2c/
[10] – https://embarcados.com.br/barramento-can-entre-arduinos-uno/

IMD0904 –Plataforma de Hardware para Internet das Coisas

Prof. Daniel Lopes Martins
daniel.martins@imd.ufrn.br

UFRN | metrópole DIGITAL